

# MIT 6.86x: 데이터 압축과 추천의 원리

## Unit 4 (Part B): Dimension Reduction & Matrix Factorization

### Contents

|     |                                                   |   |
|-----|---------------------------------------------------|---|
| 1   | Unit 4 (Part B). 차원 축소와 행렬 분해                     | 2 |
| 1.1 | 1. PCA의 머신러닝적 접근: 재구성 오차 . . . . .                | 2 |
| 1.2 | 2. 추천 시스템: 행렬 분해 (Matrix Factorization) . . . . . | 3 |
| 2   | 실전 시나리오: 넷플릭스 영화 추천                               | 4 |
| 3   | 자주 묻는 질문 (FAQ)                                    | 4 |

---

### Course Structure & Current Focus

- Unit 4 (Part A): Clustering (비슷한 것끼리 묶기)
- **Unit 4 (Part B): Dimension Reduction & Matrix Factorization**
  - 12.1 PCA: Reconstruction Error View (데이터 압축)
  - 12.2 Matrix Completion Problem (빈칸 채우기)
  - 12.3 Matrix Factorization (추천 시스템의 엔진)
  - 12.4 Collaborative Filtering (협업 필터링)
- Unit 5: Generative Models

# 1 Unit 4 (Part B). 차원 축소와 행렬 분해

지난 파트(*Clustering*)에서는 데이터를 그룹핑하여 구조를 파악했습니다. 이번에는 데이터 자체를 '압축'하거나, 비어 있는 데이터를 '복원'하는 방법을 배웁니다. 이는 마치 저화질 이미지를 고화질로 복원하거나, 내가 보지 않은 영화의 평점을 예측하는 마법 같은 기술의 기초가 됩니다.

## □ 개요 (Overview)

이 단원에서는 PCA(주성분 분석)를 '분산 최대화'가 아닌 '재구성 오차 최소화'라는 머신러닝 관점에서 재해석합니다. 또한, 넷플릭스 추천 시스템의 근간이 되는 행렬 분해 (Matrix Factorization)를 통해 희소 행렬(Sparse Matrix)의 빈칸을 채우고 사용자 취향을 예측하는 원리를 학습합니다.

## □ 핵심 용어 사전

| 용어 (Term)               | 직관적 의미 (Meaning)                          |
|-------------------------|-------------------------------------------|
| Reconstruction Error    | 원본 → 압축 → 복원했을 때, 원본과 얼마나 달라졌는가?          |
| Autoencoder             | PCA의 딥러닝 버전. 입력을 그대로 출력하도록 학습하는 신경망.      |
| Sparse Matrix           | 대부분이 0이나 비어있는(NaN) 행렬. (유저는 대부분의 영화를 안 봄) |
| Latent Factor           | 데이터 뒤에 숨어있는 진짜 원인. (예: 장르 선호도, 영화의 분위기)   |
| Collaborative Filtering | "너랑 비슷한 사람이 이거 좋아했으니 너도 좋아할 거야."          |

## 1.1 1. PCA의 머신러닝적 접근: 재구성 오차

### □ 개념 1: 파일 압축과 해제

**한 줄 요약:** 좋은 압축이란, 압축을 풀었을 때 원본과 가장 똑같은 상태여야 합니다. 손실(Error)을 최소화하는 압축 방법을 찾습니다.

### 1) 통계학 vs 머신러닝 관점 비교

- 통계학 (18.6501): "데이터가 가장 넓게 퍼진(분산이 큰) 방향을 찾자."
- 머신러닝 (6.86x): "데이터를 저차원 평면에 수직으로 내리꽂았을 때(투영), 이동 거리가 가장 짧은(오차가 적은) 평면을 찾자."

결국 피타고라스 정리에 의해 두 목표는 수학적으로 동치입니다. (분산 최대화  $\iff$  재구성 오차 최소화)

## 2) 목적 함수 (Objective Function)

$$\text{Minimize } J = \frac{1}{N} \sum_{n=1}^N \|x_n - \hat{x}_n\|^2$$

- $x_n$ : 원본 데이터 (고차원)
- $\hat{x}_n$ : 압축했다가 다시 복원한 데이터 (재구성된 값)
- 의미: 원본과 복원본 사이의 거리 제곱 합을 최소화하는 축 (Principal Component) 을 찾습니다.

## 1.2 2. 추천 시스템: 행렬 분해 (Matrix Factorization)

### □ 개념 2: 빈칸 채우기 퍼즐

**한 줄 요약:** 거대한 구멍 송송 뚫린 행렬을 두 개의 꽤 찬 작은 행렬의 곱으로 쪼개서, 빈칸의 값을 추론합니다.

### 1) 문제 상황: 희소 행렬 (Sparse Matrix)

넷플릭스에 영화가 1만 개 있다고 칩시다. 한 유저가 본 영화는 기껏해야 100개입니다. 평점 행렬  $R$ 은 99%가 비어 있습니다 (NaN). 우리의 목표는 이 빈칸에 들어갈 예상 평점을 맞추는 것입니다.

### 2) 핵심 원리: $R \approx U \times V^T$

평점 행렬  $R$  ( $N \times M$ )을 두 개의 행렬로 분해합니다.

- $U$  (User Matrix,  $N \times K$ ): 유저들의 잠재 취향 (예: 액션 선호도, 로맨스 선호도...)
- $V$  (Item Matrix,  $M \times K$ ): 영화들의 잠재 특성 (예: 액션 지수, 로맨스 지수...)

### 3) 예측 (Prediction)

유저  $u$ 가 영화  $i$ 에 줄 예상 평점  $\hat{r}_{ui}$ 는 두 잠재 벡터의 **내적(Dot Product)**입니다.

$$\hat{r}_{ui} = \mathbf{u}_u \cdot \mathbf{v}_i = \sum_{k=1}^K u_{uk} v_{ik}$$

- **직관:** (내 액션 선호도  $\times$  영화의 액션성) + (내 로맨스 선호도  $\times$  영화의 로맨스성) = 총점
- 취향과 특성이 일치할수록 (내적 값이 클수록) 높은 평점이 예측됩니다.

#### 4) 최적화 (Optimization)

실제 평점이 있는 데이터에 대해서만 오차를 줄이도록 학습합니다.

$$\min_{U,V} \sum_{(u,i) \in \text{Observed}} (r_{ui} - \mathbf{u}_u \cdot \mathbf{v}_i)^2 + \lambda(\|U\|^2 + \|V\|^2)$$

주로 **ALS (Alternating Least Squares)** 알고리즘을 사용하여  $U$ 를 고정하고  $V$ 를 풀고,  $V$ 를 고정하고  $U$ 를 푸는 방식을 반복합니다.

## 2 실전 시나리오: 넷플릭스 영화 추천

### □ Scenario: 당신이 아직 안 본 '명작' 찾기

당신은 넷플릭스 엔지니어입니다. 유저 A가 '아이언맨', '어벤저스'에는 5점을 줬고, '노트북'에는 평점을 안 남겼습니다. A에게 '노트북'을 추천해야 할까요?

#### 1. 잠재 요인 학습 ( $K = 2$ 라고 가정):

- 차원 1: [액션/폭파], 차원 2: [감동/로맨스]
- 행렬 분해를 해보니, 유저 A의 벡터  $u_A = [5.0, 0.2]$  (액션광)
- 영화 '노트북'의 벡터  $v_{Nb} = [0.1, 4.8]$  (완전 로맨스)

#### 2. 예상 평점 계산 (내적):

$$\hat{r} = (5.0 \times 0.1) + (0.2 \times 4.8) = 0.5 + 0.96 = 1.46$$

3. 결정: 예상 평점이 1.46점으로 매우 낮습니다. → 추천하지 않습니다. (대신 액션 지수가 높은 '트랜스포머'를 추천합니다.)

## 3 자주 묻는 질문 (FAQ)

**Q1. 잠재 요인(Latent Factor)이 무엇인지 컴퓨터가 알려주나요?** A. 아니요, 컴퓨터는 그냥 숫자 벡터만 줍니다.  $k = 1$  번째 차원이 "액션"인지 "공포"인지는 사람이 결과를 보고 해석해야 합니다. 때로는 해석 불가능한 추상적인 특징일 수도 있습니다.

**Q2. 콜드 스타트(Cold Start) 문제는 뭔가요?** A. 새로 가입한 유저나 막 개봉한 영화는 데이터(평점)가 하나도 없어서  $U$ 나  $V$  벡터를 학습할 수 없는 문제입니다. 이 경우엔 협업 필터링 대신, 나이/성별 기반의 베스트셀러 추천(Top-N)이나 콘텐츠 기반 필터링을 섞어서 사용합니다.

**Next Step:** 지금까지는 주어진 데이터를 분석하는 데 집중했습니다. 그런데 AI가 데이터를 분석하는 것을 넘어, 새로운 데이터를 창조할 수는 없을까요? 다음 **Unit 5: 생성 모델 (Generative Models)**에서는 데이터를 직접 만들어내는 VAE와 GAN에 대해 배웁니다.

#### Unit 4 (Part B) 핵심 요약

- **PCA:** 데이터의 재구성 오차를 최소화하는 평면을 찾아 차원을 축소한다.
- **행렬 분해 (MF):**  $R \approx UV^T$ . 희소 행렬을 분해하여 숨겨진 패턴을 찾는다.
- **협업 필터링:** 유저와 아이템의 잠재 벡터 내적을 통해 선호도를 예측한다.
- **활용:** 넷플릭스, 유튜브, 쇼핑몰 등의 개인화 추천 시스템의 핵심 원리.